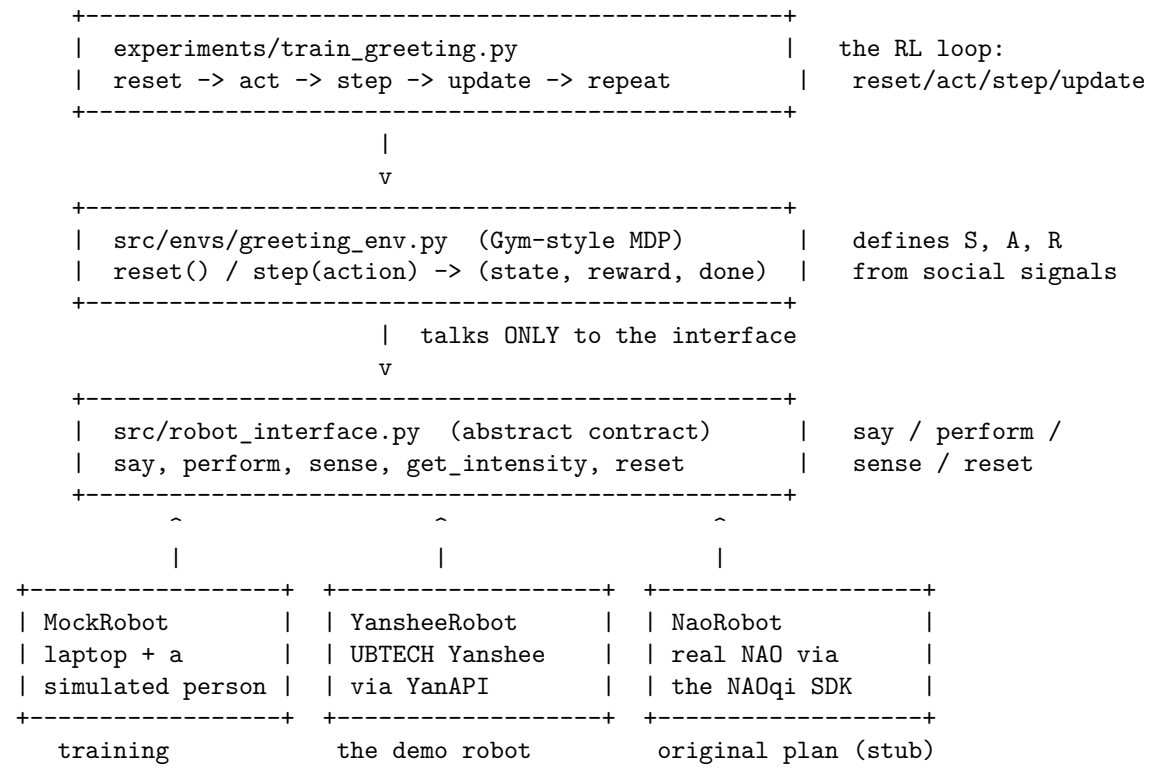
Architecture

The whole point of this codebase is one idea: **the reinforcement-learning loop should not know or care what robot it is driving.** That lets us prototype on a laptop today and move to a (virtual or real) NAO later with a one-line change.

Layers



The NAO was the original target; the robot available for the demo is a Yanshee, so `YansheeRobot` was added and `NaoRobot` stays as a documented stub.

Why this shape

- **The agent and environment are robot-agnostic.** They import `RobotInterface`, never `MockRobot` or `NaoRobot`. Swapping implementations changes one line in the experiment script.
- **The reward comes from social signals**, not from privileged knowledge. The `MockRobot` hides the person's comfort preference; the agent must discover it from head-touch ("approve") and the person leaving ("crowded") signals — the same signals a real NAO would read from its tactile sensors and sonar.
- **Sensor noise is injected in the mock** so the learned policy is not brittle. This shrinks the eventual sim-to-real ("reality") gap: a policy that already copes with noisy simulated sonar transfers better to noisy real sonar.

Mapping to NAOqi (real NAO)

Interface method	NAOqi call (real NAO)
`say(text)`	`ALTextToSpeech.say(text)`

Interface method	NAOqi call (real NAO)
`perform("closer")`	`ALMotion.moveTo(0.30, 0, 0)`
`perform("back")`	`ALMotion.moveTo(-0.30, 0, 0)`
`sense().distance`	`ALMemory` ultrasonic keys (`.../US/Left|Right/...`)
`sense().touch`	`ALMemory` head-tactile keys (`FrontTactilTouched` …)
`sense().present`	`ALFaceDetection`

Mapping to YanAPI (Yanshee, the demo robot)

Interface method	YanAPI call (see `CALLS` in `src/yanshee_robot.py`)
`say(text)`	`sync_do_tts(text, True)`
`perform("closer")`	`sync_play_motion(name="walk", direction="forward")`
`perform("back")`	`sync_play_motion(name="walk", direction="backward")`
`perform("intensity_*")`	`set_robot_volume(...)`, a gesture, and a spoken greeting
`sense().distance`	`get_sensors_infrared_value()`, scaled by 1/3 to a tabletop
`sense().touch`	touch sensor (name confirmed in the robot session)
`sense().present`	infrared reads nobody within 1.2 m

Two things differ from the simulation. The robot cannot re-home itself, so the operator places it at the start of each interaction. And it will not walk closer than about 10 cm, where the simulation simply clamps the distance.

Reusing the skeleton for other experiments

The greeting/proxemics task is illustrative. The other candidate directions reuse the same loop and interface; only the bindings change:

Direction	State features	Actions	Reward signal
Greeting / proxemics	distance bin, intensity	move, intensity, hold	touch, presence
Tutoring / pace	recent correctness, latency	speed up/down, easier/harder	correctness, engagement
Engagement maximization	engagement level (gaze)	pick behaviour from repertoire	change in gaze / proximity